LAB 7 – FUZZING FOR INPUT VALIDATION BUGS

Project: L6_L7_13 – Pet Walking Service
Experiment: Lab 7 – Fuzzing for Input Validation Bugs
Student Name: D.PRANAV SAI
SRN : PES1UG24CS139

---

1. OBJECTIVE

To explore fuzz testing as a security and quality assurance technique by identifying and fixing input validation bugs in a Python application using the Hypothesis fuzzing library.

The objectives are:

- Understand the purpose and methodology of fuzz testing.
- Apply property-based fuzz testing using Hypothesis.
- Identify bugs caused by unexpected and invalid inputs.
- Improve application robustness through defensive programming.

2. SOFTWARE REQUIREMENTS

- Python 3.12.10
- Hypothesis library
- Pytest
- Visual Studio Code

3. APPLICATION OVERVIEW

The Pet Walking Service application contains three functions:

1. **parse_walk_schedule():** Parses the pet walking schedule, calculates the base fee, pet charges and weekend surcharge, and returns the total walking cost.
2. **compute_pace_per_pet():** Calculates the average walking time per pet using the total walking minutes and number of pets.
3. **lookup_temperament_surcharge():** Determines the additional surcharge based on the pet's temperament code.

4. STEP 1 – EXECUTION OF THE ORIGINAL APPLICATION

Initially, the original processor.py application was executed using:

python processor.py

The following sample inputs were provided:

- Walk schedule: 2-30-YES
- Walking metrics: 45/3

- Temperament code: CALM

The application executed successfully with the valid inputs and generated the expected results:

- Walk: 2 pets, 30 mins – Total $45.00
- Pace: 15.0 minutes per pet
- Temperament surcharge: $0.0

```
D:\L6_L7_13_pet_walking_service-main\Lab 7>

D:\L6_L7_13_pet_walking_service-main\Lab 7>python processor.py
==== Running parse_walk_schedule ====
Enter schedule (e.g. 2-30-YES): 2-30-YES
Walk: 2 pets, 30 mins - Total $45.00

==== Running compute_pace_per_pet ====
Enter minutes/pets (e.g. 45/3): 45/3
Pace (min/pet): 15.0

==== Running lookup_temperament_surcharge ====
Enter temperament code (e.g. CALM, PLAYFUL, AGGRESSIVE): CALM
Surcharge: $ 0.0

D:\L6_L7_13_pet_walking_service-main\Lab 7>
```

5. STEP 2 – IMPLEMENTATION OF HYPOTHESIS FUZZ TESTING

The required dependencies were installed using:

pip install hypothesis pytest

The test_processor.py file was completed using the Hypothesis library.

Three test functions were implemented:

1. test_parse_walk_schedule_no_crash()
2. test_compute_pace_per_pet_safe()
3. test_lookup_temperament_surcharge_safe()

The strategy st.text() | st.none() was used to generate random strings and None values.

The @given decorator was used to automatically generate test inputs. Try-except blocks were implemented to identify unexpected exceptions.

6. STEP 3 – IDENTIFICATION OF BUGS

The tests were executed using:

python -m pytest test_processor.py -v

Initially, all three tests failed, revealing multiple input validation problems.

6.1 Bug in parse_walk_schedule()

Hypothesis identified the following problems:

- AttributeError when the input was None.
- ValueError when the schedule contained invalid integer values.
- ValueError when the schedule did not contain exactly three fields.

Reason: The original function assumed that the input would always contain three correctly formatted fields with valid numerical values.

6.2 Bug in compute_pace_per_pet()

Hypothesis identified the following problems:

- AttributeError when the input was None.
- ValueError when the input did not contain the required / separator.
- ValueError when invalid numerical values were provided.
- Potential ZeroDivisionError when the number of pets was zero.

Reason: The original function did not validate the input format or check whether the number of pets was zero before performing division.

6.3 Bug in lookup_temperament_surcharge()

Hypothesis identified the following problems:

- AttributeError when the input was None.
- KeyError when an empty or unknown temperament code was provided.

Reason: The original function directly accessed dictionary values without validating the input or checking whether the temperament code existed.

```
D:\L6_L7_13_pet_walking_service-main\Lab 7>python -m pytest test_processor.py -v
============================= test session starts =============================
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\koush\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: D:\L6_L7_13_pet_walking_service-main\Lab 7
plugins: anyio-4.15.1, hypothesis-6.170.15, platformdirs-4.13.0
collected 3 items

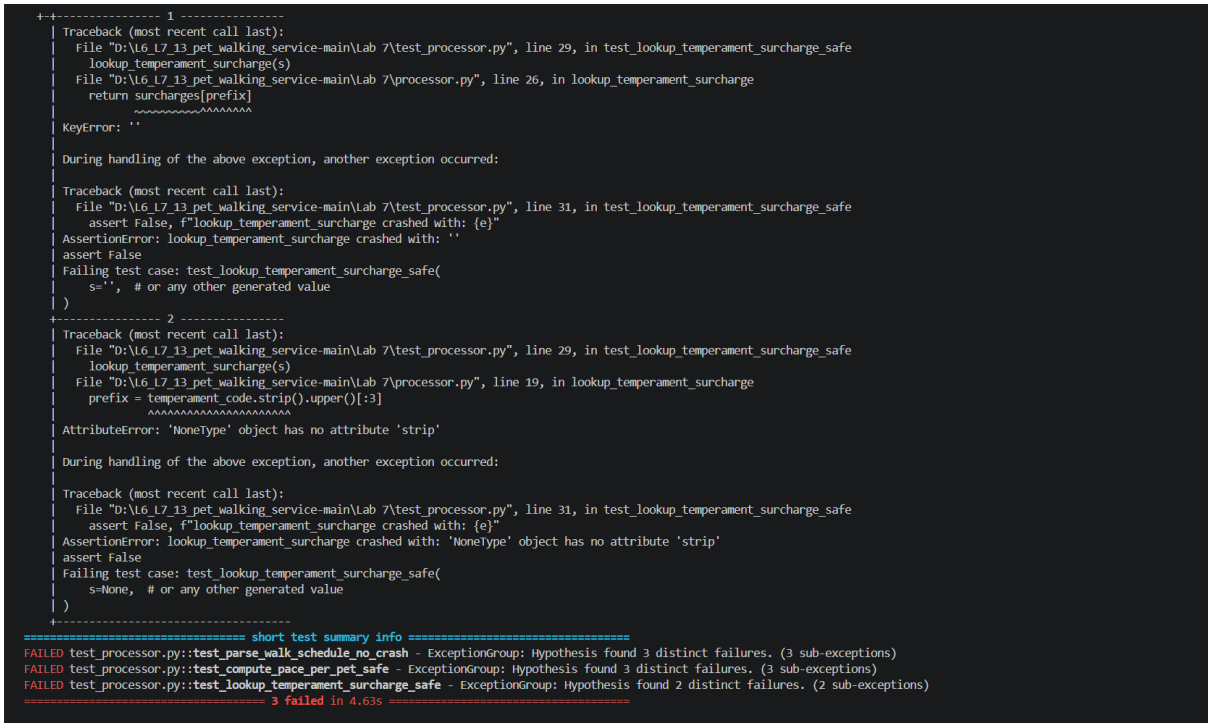
test_processor.py::test_parse_walk_schedule_no_crash FAILED              [ 33%]
test_processor.py::test_compute_pace_per_pet_safe FAILED                 [ 66%]
test_processor.py::test_lookup_temperament_surcharge_safe FAILED         [100%]

================================== FAILURES ===================================
_____________________ test_parse_walk_schedule_no_crash _____________________
  + Exception Group Traceback (most recent call last):
  |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\test_processor.py", line 11, in test_parse_walk_schedule_no_crash
  |     def test_parse_walk_schedule_no_crash(s):
  |                ^^^
  |   File "C:\Users\koush\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\LocalCache\local-packages\Python312\site-packages\hypothesis\core.py", line 2334, in wrapped_test
  |     raise the_error_hypothesis_found
  | ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\test_processor.py", line 13, in test_parse_walk_schedule_no_crash
    |     parse_walk_schedule(s)
    |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\processor.py", line 3, in parse_walk_schedule
    |     pets = int(pets_str)
    |            ^^^^^^^^^^^^
    | ValueError: invalid literal for int() with base 10: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\test_processor.py", line 15, in test_parse_walk_schedule_no_crash
    |     assert False, f"parse_walk_schedule crashed with: {e}"
    | AssertionError: parse_walk_schedule crashed with: invalid literal for int() with base 10: ''
    | assert False
    | Failing test case: test_parse_walk_schedule_no_crash(
    |     s='--',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
```

```
    |     pets = int(pets_str)
    |            ^^^^^^^^^^^^
    | ValueError: invalid literal for int() with base 10: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\test_processor.py", line 15, in test_parse_walk_schedule_no_crash
    |     assert False, f"parse_walk_schedule crashed with: {e}"
    | AssertionError: parse_walk_schedule crashed with: invalid literal for int() with base 10: ''
    | assert False
    | Failing test case: test_parse_walk_schedule_no_crash(
    |     s='--',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\test_processor.py", line 13, in test_parse_walk_schedule_no_crash
    |     parse_walk_schedule(s)
    |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\processor.py", line 2, in parse_walk_schedule
    |     pets_str, mins_str, weekend_str = schedule_spec.strip().split('-')
    |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    | ValueError: not enough values to unpack (expected 3, got 1)
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\test_processor.py", line 15, in test_parse_walk_schedule_no_crash
    |     assert False, f"parse_walk_schedule crashed with: {e}"
    | AssertionError: parse_walk_schedule crashed with: not enough values to unpack (expected 3, got 1)
    | assert False
    | Failing test case: test_parse_walk_schedule_no_crash(
    |     s='',  # or any other generated value
    | )
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\test_processor.py", line 13, in test_parse_walk_schedule_no_crash
    |     parse_walk_schedule(s)
    |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\processor.py", line 2, in parse_walk_schedule
    |     pets_str, mins_str, weekend_str = schedule_spec.strip().split('-')
    |                                       ^^^^^^^^^^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'strip'
    |
    | During handling of the above exception, another exception occurred:
    |
```

```
    |
    | Traceback (most recent call last):
    |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\test_processor.py", line 15, in test_parse_walk_schedule_no_crash
    |     assert False, f"parse_walk_schedule crashed with: {e}"
    | AssertionError: parse_walk_schedule crashed with: 'NoneType' object has no attribute 'strip'
    | assert False
    | Failing test case: test_parse_walk_schedule_no_crash(
    |     s=None,  # or any other generated value
    | )
    +------------------------------------
_____________________ test_compute_pace_per_pet_safe _____________________
  + Exception Group Traceback (most recent call last):
  |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\test_processor.py", line 19, in test_compute_pace_per_pet_safe
  |     def test_compute_pace_per_pet_safe(s):
  |                ^^^
  |   File "C:\Users\koush\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\LocalCache\local-packages\Python312\site-packages\hypothesis\core.py", line 2334, in wrapped_test
  |     raise the_error_hypothesis_found
  | ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\test_processor.py", line 21, in test_compute_pace_per_pet_safe
    |     compute_pace_per_pet(s)
    |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\processor.py", line 13, in compute_pace_per_pet
    |     mins = float(mins_str)
    |            ^^^^^^^^^^^^^^
    | ValueError: could not convert string to float: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\test_processor.py", line 23, in test_compute_pace_per_pet_safe
    |     assert False, f"compute_pace_per_pet crashed with: {e}"
    | AssertionError: compute_pace_per_pet crashed with: could not convert string to float: ''
    | assert False
    | Failing test case: test_compute_pace_per_pet_safe(
    |     s='/',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\test_processor.py", line 21, in test_compute_pace_per_pet_safe
    |     compute_pace_per_pet(s)
    |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\processor.py", line 12, in compute_pace_per_pet
    |     mins_str, pets_str = walk_metrics_str.split('/')
    |     ^^^^^^^^^^^^^^^^^^
```

```
    |     mins_str, pets_str = walk_metrics_str.split('/')
    |     ^^^^^^^^^^^^^^^^^^
    | ValueError: not enough values to unpack (expected 2, got 1)
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\test_processor.py", line 23, in test_compute_pace_per_pet_safe
    |     assert False, f"compute_pace_per_pet crashed with: {e}"
    | AssertionError: compute_pace_per_pet crashed with: not enough values to unpack (expected 2, got 1)
    | assert False
    | Failing test case: test_compute_pace_per_pet_safe(
    |     s='',  # or any other generated value
    | )
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\test_processor.py", line 21, in test_compute_pace_per_pet_safe
    |     compute_pace_per_pet(s)
    |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\processor.py", line 12, in compute_pace_per_pet
    |     mins_str, pets_str = walk_metrics_str.split('/')
    |                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'split'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\test_processor.py", line 23, in test_compute_pace_per_pet_safe
    |     assert False, f"compute_pace_per_pet crashed with: {e}"
    | AssertionError: compute_pace_per_pet crashed with: 'NoneType' object has no attribute 'split'
    | assert False
    | Failing test case: test_compute_pace_per_pet_safe(
    |     s=None,  # or any other generated value
    | )
    +------------------------------------
_____________________ test_lookup_temperament_surcharge_safe _____________________
  + Exception Group Traceback (most recent call last):
  |   File "D:\L6_L7_13_pet_walking_service-main\Lab 7\test_processor.py", line 27, in test_lookup_temperament_surcharge_safe
  |     def test_lookup_temperament_surcharge_safe(s):
  |                ^^^
  |   File "C:\Users\koush\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\LocalCache\local-packages\Python312\site-packages\hypothesis\core.py", line 2334, in wrapped_test
  |     raise the_error_hypothesis_found
  | ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
```



7. STEP 4 – FIXING THE APPLICATION

The original processor.py file was modified to handle unexpected and invalid inputs.

The following improvements were implemented:

7.1 parse_walk_schedule()

- Added input type validation using isinstance().
- Checked whether the schedule contains exactly three fields.
- Validated the number of pets and walking duration.
- Ensured that pets and minutes are positive.
- Validated the weekend code against YES and NO.
- Added exception handling for invalid integer values.

7.2 compute_pace_per_pet()

- Added input type validation.
- Checked whether the input contains exactly two values separated by /.
- Added numerical validation using float conversion.
- Prevented division by zero.
- Ensured that walking minutes and pet count are positive.
- Returns None for invalid inputs.

7.3 lookup_temperament_surcharge()

- Added input type validation.
- Preserved the original uppercase conversion and prefix extraction.
- Replaced direct dictionary indexing with the .get() method.
- Returns None for unknown temperament codes instead of raising a KeyError.

These modifications ensure that unexpected inputs are handled safely without causing application crashes.

8. STEP 5 – EXECUTION OF THE FIXED APPLICATION

After modifying processor.py, the application was executed again using:

python processor.py

The following valid inputs were provided:

- Walk schedule: 2-30-YES
- Walking metrics: 45/3
- Temperament code: CALM

The application successfully generated the expected results:

- Walk: 2 pets, 30 mins – Total $45.00
- Pace: 15.0 minutes per pet
- Temperament surcharge: $0.0

The fixed application executed successfully without errors.

```
D:\L6_L7_13_pet_walking_service-main\Lab 7>python processor.py
==== Running parse_walk_schedule ====
Enter schedule (e.g. 2-30-YES): 2-30-YES
Walk: 2 pets, 30 mins - Total $45.00

==== Running compute_pace_per_pet ====
Enter minutes/pets (e.g. 45/3): 45/3
Pace (min/pet): 15.0

==== Running lookup_temperament_surcharge ====
Enter temperament code (e.g. CALM, PLAYFUL, AGGRESSIVE): CALM
Surcharge: $ 0.0
```

9. STEP 6 – RE-EXECUTION OF HYPOTHESIS TESTS

After fixing the application, the Hypothesis tests were executed again using:

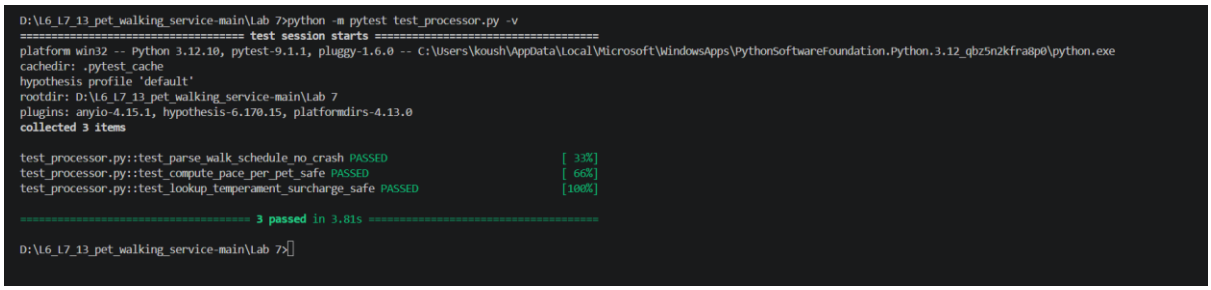
python -m pytest test_processor.py -v

The final test results were:

- test_parse_walk_schedule_no_crash – PASSED
- test_compute_pace_per_pet_safe – PASSED
- test_lookup_temperament_surcharge_safe – PASSED

Total: 3 passed

This confirms that all three functions handled the automatically generated test inputs without raising unhandled exceptions.



10. STEP 7 – MANUAL INVALID INPUT VERIFICATION

Additional manual tests were performed to verify that the modified functions handle invalid inputs safely.

The following statements were executed:

print(parse_walk_schedule(None))

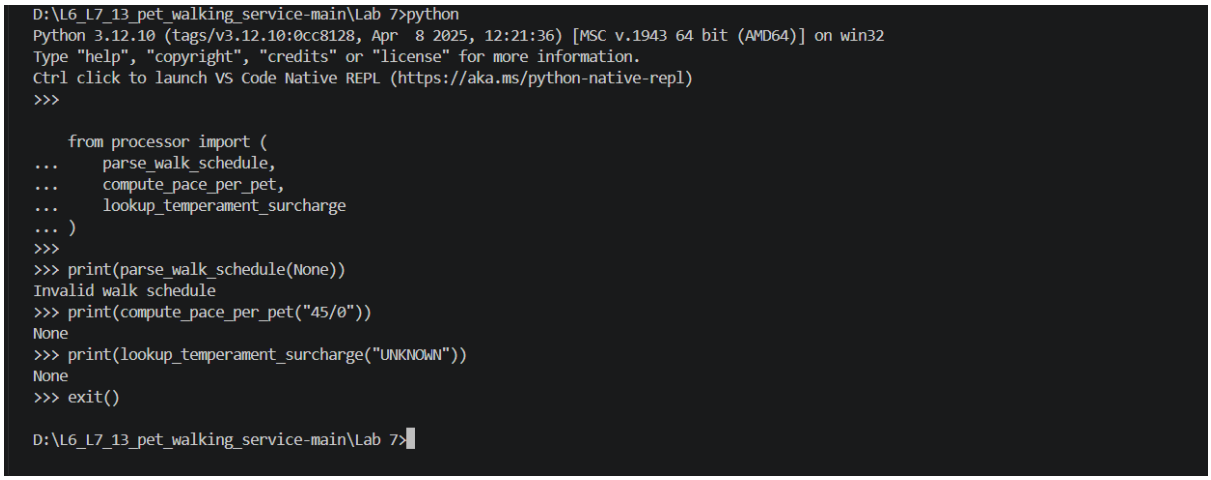
print(compute_pace_per_pet("45/0"))

print(lookup_temperament_surcharge("UNKNOWN"))

Expected results:

Test input	**Expected output**
None for walk schedule	Invalid walk schedule
45/0 for walking metrics	None
UNKNOWN for temperament	None

These tests demonstrate that the application handles invalid schedules, prevents division by zero and safely handles unknown temperament codes.



11. CONCLUSION

The lab successfully demonstrated the application of fuzz testing to identify input validation bugs in the Pet Walking Service Python application.

Using the Hypothesis library, automatically generated inputs revealed several bugs in all three application functions.

The identified problems were fixed by implementing input validation, exception handling and defensive programming techniques.

After modifying the application, all three Hypothesis tests passed successfully. The application also produced the expected results with valid inputs.

Therefore, the experiment demonstrated how property-based fuzz testing can improve the reliability, robustness and input-handling capabilities of software applications.